Example Backend Implementation

Python + SQLite using `student_portal.db`

This example shows:

- backend functions
- SQL executed safely
- clear separation of responsibility
- reuse of logic

Final Database Reminder (What We're Using)

Tables

users

- id (PRIMARY KEY)
- username (UNIQUE)
- email (UNIQUE)

records

- id (PRIMARY KEY)
- user_id (FOREIGN KEY → users.id)
- score

Project Structure (Simple & Professional)

```
sql_python_backend/
│
├── database_app.py
└── student_portal.db
```

Only one file for now. Clean and beginner-friendly.

Full Backend Example (`database_app.py`)

This is a **complete working file**.

Students can:

- run it
- read it top to bottom
- clearly see what backend code does

🔹 Step A: Connect to the Database

```
import sqlite3

def get_connection():
    return sqlite3.connect("student_portal.db")
```

Why this exists:

- One place to manage database connections
- Reusable across functions

🔹 Step B: Create Tables (Run Once, Safe to Re-run)

```
def create_tables(cursor):
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        username TEXT NOT NULL UNIQUE,
        email TEXT NOT NULL UNIQUE
    )
    """)

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS records (
        id INTEGER PRIMARY KEY,
        user_id INTEGER,
        score INTEGER NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id)
    )
    """)
```

🔹 Step C: Backend Logic – Create User

```
def create_user(cursor, username, email):
    cursor.execute(
        "INSERT INTO users (username, email) VALUES (?, ?)",
```

```
        (username, email)
    )
```

Key backend idea:

- No input()
- No print()
- SQL placeholders used

🔹 Step D: Backend Logic – Add Record (Score)

```
def add_record(cursor, user_id, score):
    cursor.execute(
        "INSERT INTO records (user_id, score) VALUES (?, ?)",
        (user_id, score)
    )
```

🔹 Step E: Backend Logic – Get All Users

```
def get_all_users(cursor):
    cursor.execute("SELECT * FROM users")
    return cursor.fetchall()
```

🔹 Step F: Backend Logic – Get User Scores (JOIN)

```
def get_user_scores(cursor):
    cursor.execute("""
    SELECT users.username, records.score
    FROM users
    JOIN records ON users.id = records.user_id
    """)
    return cursor.fetchall()
```

This **proves**:

- normalisation works
- relationships work
- backend can combine data safely

🔹 Step G: Main Program (Execution Layer)

```
def main():
```

```
    connection = get_connection()
    cursor = connection.cursor()

    create_tables(cursor)

    create_user(cursor, "alice", "alice@email.com")
    create_user(cursor, "bob", "bob@email.com")

    add_record(cursor, 1, 85)
    add_record(cursor, 2, 90)

    connection.commit()

    users = get_all_users(cursor)
    print("Users:")
    for user in users:
        print(user)

    scores = get_user_scores(cursor)
    print("\nUser Scores:")
    for score in scores:
        print(score)

    connection.close()
```

◆ Step H: Run the Program

At the bottom of the file:

```
if __name__ == "__main__":
    main()
```

Expected Output (Terminal)

```
Users:
(1, 'alice', 'alice@email.com')
(2, 'bob', 'bob@email.com')

User Scores:
('alice', 85)
('bob', 90)
```